Двоичные деревья поиска

Плотников Даниил Михайлович

Санкт-Петербургский государственный университет

Оглавление

1. Двоичное дерево поиска 3

1.1 Интуиция 4

1.4 Структура данных 7

1.6 Вставка 9

1.8 Поиск 11

2. AVL деревья 20

2.1 Баланс в AVL 21

2.2 Баланс-фактор 22

2.3 Вращения 23

Двоичное дерево поиска

Интуиция

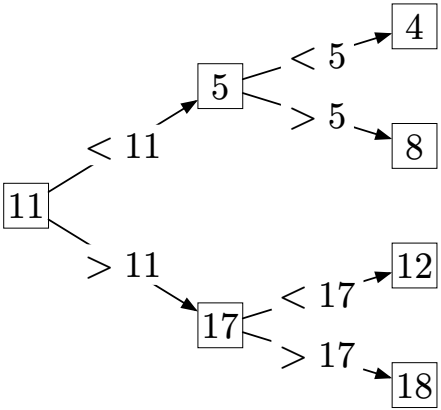
Представим обычный остортированный массив и задачу о поиске элемента в этом массиве. Эффективное решение довольно очевидно: использовать двоичный поиск. Но что если нам нужно иметь возсожность добавлять/удалять элементы из этого массива? Эффективный поиск за $O(\log n)$ будет негативно сбалансирован добавлением/удалением за $O(n)$.

Попробуем изменить наш подход. Рассмотрим как выглядит операции поиска в этом массиве.

4	5	8	11	12	17	18

4	5	8	11	12	17	18

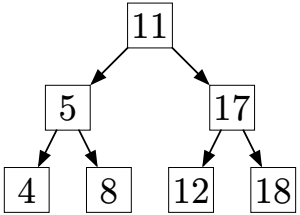
Ниже расположены все возможные пути двоичного поиска на массиве.



А что если мы запишем массив как это дерево? Рассмотрим его свойства.

Рассмотрим какие свойства имеет это дерево по построению:

1. Левый потомок и все его потомки будут иметь значение меньше чем текущая вершина.
2. Правый потомок и все его потомки будут иметь значение больше чем текущая вершина.
3. Глубина дерева не будет превышать $\log n$, где n — количество вершин



Значит, опираясь на эти свойства, мы можем однозначно найти место для вставки и вставить новый элемент за $O(\log n)$, а так же проверить наличие и удалить элемент за те же $O(\log n)$

Структура данных

```
struct Node {
    int value;
    Node* left;
    Node* right;

    Node(int v) : value(v), left(nullptr), right(nullptr) {}
};
```

Вставка

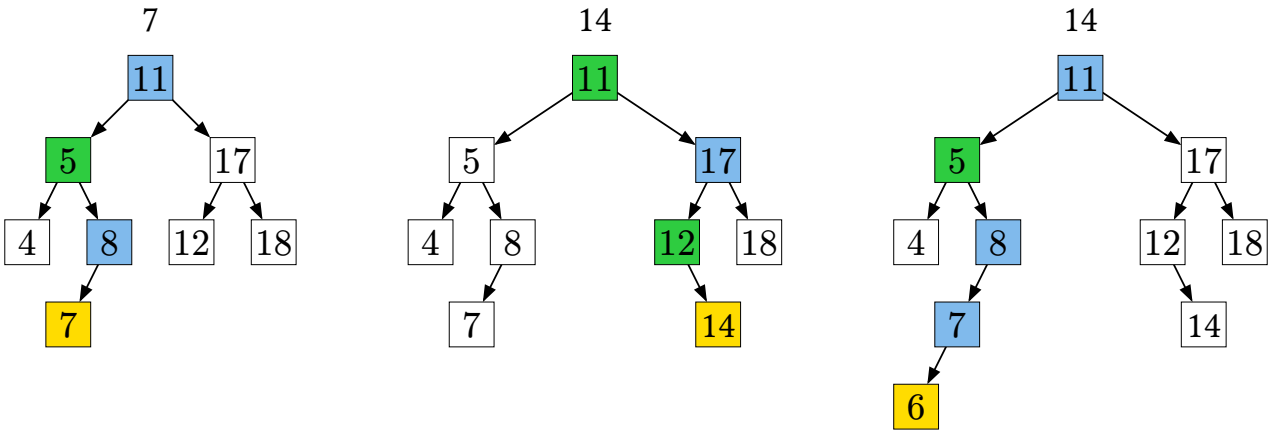
```
Node* insert(Node* node, int value) {
    if (node == nullptr) {
        return new Node(value);
    }
    if (value < node->value) {
        node->left = insert(node->left, value);
    } else {
        node->right = insert(node->right, value);
    }
    return node;
}
```

Поиск

```
Node* search(Node* node, int value) {
    if (node == nullptr || node->value == value) {
        return node;
    }
    if (value < node->value) {
        return search(node->left, value);
    }
    return search(node->right, value);
}
```

Попробуем добавить элементы 7, 14 и 6

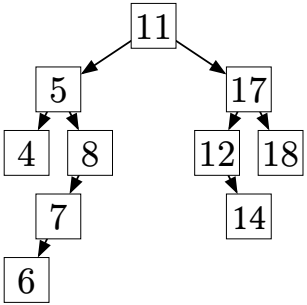
На диаграммах ниже зеленая вершина обозначает что наше значение больше текущей вершины, значит спуск совершается вправо. Синий значит действие противоположное. Желтым помечена текущая вершина.



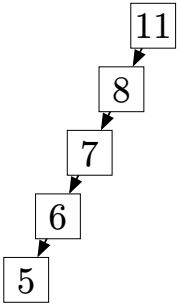
Снова рассмотрим свойства:

Свойства 1,2 мы сохранили успешно.

Однако, со свойством 3 есть существенная проблема. Путь $11 \to 5 \to 7 \to 7 \to 6$ имеет длину 5, однако количество вершин 10, $\log_2 10 \approx 4$. В худшем случае дерево станет лозой, и тогда все операции будут $O(n)$. Решение этой проблемы называется балансировкой.



Пример лозы(также называют бамбуком):



Структура данных

```
struct Node {
    int value;
    Node* left;
    Node* right;

    Node(int v) : value(v), left(nullptr), right(nullptr) {}
};
```

Вставка

```
Node* insert(Node* node, int value) {
    if (node == nullptr) {
        return new Node(value);
    }
    if (value < node->value) {
        node->left = insert(node->left, value);
    } else {
        node->right = insert(node->right, value);
    }
    return node;
}
```

Поиск

```
Node* search(Node* node, int value) {
    if (node == nullptr || node->value == value) {
        return node;
    }
    if (value < node->value) {
        return search(node->left, value);
    }
    return search(node->right, value);
}
```

Удаление

```
Node* remove(Node* node, int value) {
    if (node == nullptr) return nullptr;

    if (value < node->value) {
        node->left = remove(node->left, value);
    } else if (value > node->value) {
        node->right = remove(node->right, value);
    } else {
        // Нашли узел для удаления
        // ...
    }
    return node;
}
```

```
// Случай: нет левого ребёнка
if (node->left == nullptr) {
    Node* temp = node->right;
    delete node;
    return temp;
}
// Случай: нет правого ребёнка
if (node->right == nullptr) {
    Node* temp = node->left;
    delete node;
    return temp;
}
```

```
// Случай: два ребёнка
// Ищем минимальный элемент в правом поддереве
Node* minLarger = node->right;
while (minLarger->left != nullptr) {
    minLarger = minLarger->left;
}
node->value = minLarger->value;
node->right = remove(node->right, minLarger->value);
```

AVL деревья

Баланс в AVL

AVL деревья поддерживают следующий инвариант: для каждой вершины $\mathrm{BF} \leq 1$, где $\mathrm{BF} = h(\mathrm{left}) - h(\mathrm{right})$

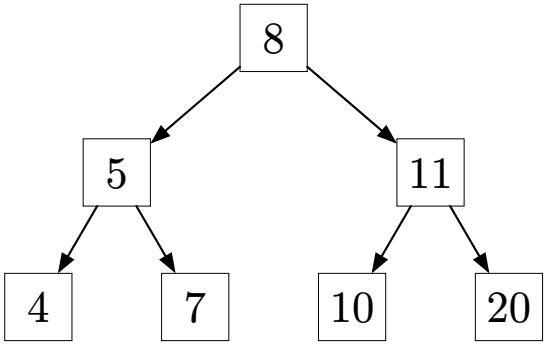
Почему это важно?

Если высота дерева $h = O(\log n)$, то все операции работают за $O(\log n)$.

Как поддерживается?

После каждой вставки/удаления поднимаемся обратно и пересчитываем высоты. Если $|\mathrm{BF}| > 1$, выполняем вращение.

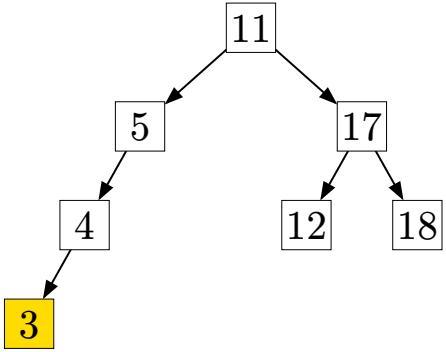
Пустые поддеревья имеют высоту 0.



Баланс-фактор

Для каждой вершины храним высоту поддерева:

11	5	17	12	18
2	1	1	0	0



Баланс-факторы:

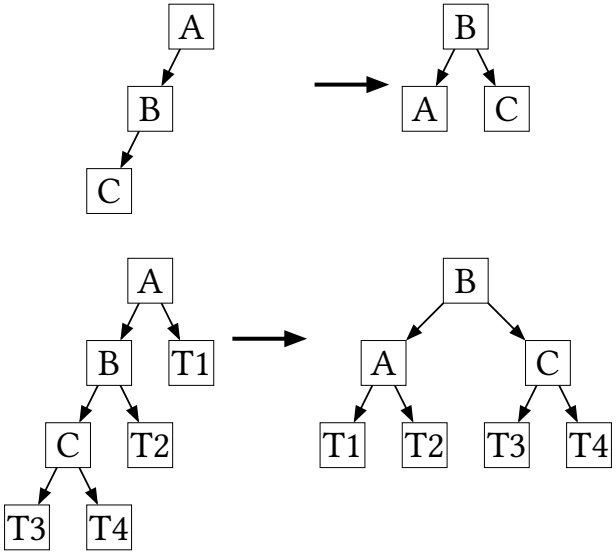
- $11 : h(L) - h(R) = 2 - 2 = 0$
- $5 : h(L) - h(R) = 1 - 0 = 1$
- $17 : h(L) - h(R) = 1 - 1 = 0$

Если добавить 3 как левый ребёнок 4:

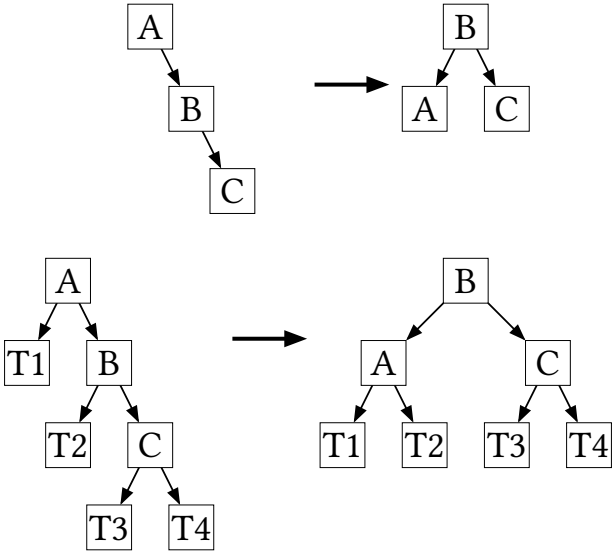
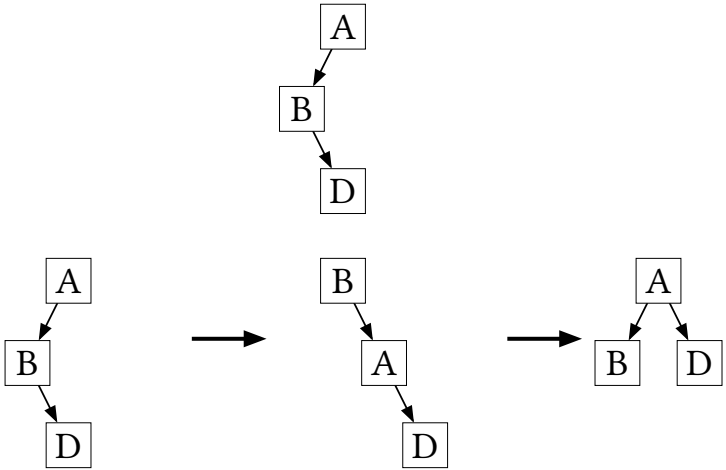
- $4 : h(L) - h(R) = 1 - 0 = 1$
- $5 : h(L) - h(R) = 2 - 0 = 2$ Нарушен!

Вращения

Правое вращение



Левое вращение



Всегда ли верно?